Epoch — Project Status

Current state of design, website, application and release

Version: 0.1.0.dev0
Date: 2026-10-20
Status: Development build available; no stable release

Table of Contents

Table of Contents 2

1. Summary 3

Rename to Epoch (session 4) 4

2. Website (16 pages) 5

3. Application 6

3.1 Working functionality 6

3.2 Packaging 6

4. Verified build and test results (2026-10-20) 7

5. Known defects and limitations 8

6. Not done 9

7. Key decisions 10

8. Next recommended actions 11

1. Summary

Area	State
Technical design PDF	**Complete** (27 pages, `docs/`)
Website	**Complete as a static site (16 pages); not deployed**
Release manifest and downloads	**Complete**; one real development-channel zip, verified
Tabular training engine	**Complete for v0.1 scope** (CSV classification and regression)
Desktop front end (PySide6)	**Complete for v0.1 scope**: 12 screens, results chart, demo dataset, threaded import; all screens reviewed in native-render screenshots
Windows packaging	**Partial**: unsigned zip with windowed GUI launcher and console CLI launcher; no installer
Other domains, HPO, CV, GPU, plugins	**Planned only**

Environment: Windows 11, Python 3.12.4, no git, no GPU tooling.

Rename to Epoch (session 4)

- The product is now **Epoch**. All user-facing text, the website, docs, PDFs, window title, launchers (`Epoch.exe`, `EpochCLI.exe`), zip name and the workspace database name (`epoch.sqlite`; old `engine_trainer.sqlite` workspaces still open) were renamed. The internal Python package is still `engine_trainer`.
- Zip rebuilt as `Epoch-0.1.0.dev0-win-x64.zip` (238,311,200 bytes); frozen `EpochCLI.exe --selftest` OK; GUI launcher alive after 10 s; 31 app tests, 8 site tests, link check all pass.
- **Logo: not applied.** The owner said it was uploaded under `website/src/assets`, but no logo file exists there or anywhere in the project (only `favicon.svg`, the placeholder mark made in session 1). `website/build.py` now uses `src/assets/logo.svg|png|webp` automatically if present. The desktop app and favicon still use the placeholder.

2. Website (16 pages)

Home, Features, How it works, Downloads, Documentation index plus 9 docs (installation, quick start, dataset preparation, training workflow, model export, troubleshooting, system requirements, supported features, FAQ), Changelog and roadmap, About and contact.

- Built by `website/build.py` from `releases/releases.json`, `src/pages`, `src/docs`, `src/data/roadmap.json`.
- A download button appears only if the artifact file exists and matches its recorded SHA-256 and size (or a URL has a recorded verification time). Unavailable artifacts show their reason instead.
- Home and Features use real screenshots of the development build (synthetic demo data, labelled as such).
- Mobile width check (375 px): no horizontal overflow on the pages measured.
- **Not deployed.** Deployment steps are in `README.md`. No source repository or support channel exists, and the site says so.

3. Application

```
app/engine_trainer/
  io_csv.py profile.py validate.py splits.py pipeline.py candidates.py evaluate.py
  runner.py   pure run_plan(): train on TRAIN, score on VAL, never touches TEST
  jobs.py     separate spawn process, event queue, CPU/RAM sampling, real cancel
  store.py    SQLite schema      workspace.py   service facade (GUI + CLI)
  export.py inference.py   export bundle + standalone loader (explicit trust flag)
  cli.py      train | profile | --selftest | gui
  gui/app.py gui/screens.py   main window and 12 screens, ImportWorker thread, BarChart
```

3.1 Working functionality

- CSV import in a background thread (encoding/delimiter detection, limits, duplicate-header check, immutable copy with SHA-256)
- Synthetic demo dataset button (generated data, clearly named)
- Profiling, target suggestion, task inference that asks when ambiguous
- Validation findings (errors block), leakage probe on training rows, imbalance and duplicate warnings
- Seeded stratified split, preprocessing fit on train only
- Candidates: baseline, logistic/ridge, random forest, histogram gradient boosting; worker process; live per-candidate progress; CPU/RAM readout; cancel kills the worker
- Results: bar chart against the baseline, metrics table, confusion matrix, leakage warnings
- Single-use test evaluation with confirmation on repeat; SQLite history and registry
- Export bundle (pipeline, schema, labels, config, metrics, requirements, inference code, manifest); defaults to `<workspace>/exports/<model>`
- Planned features appear as purple "planned" lines, not as controls

3.2 Packaging

`python tools/build_release.py` builds a PyInstaller one-folder app with two launchers: `Epoch.exe` (windowed) and `EpochCLI.exe` (console). The script runs `EpochCLI.exe --selftest`, confirms the GUI launcher stays alive for 10 s, zips and hashes. It never edits the manifest.

4. Verified build and test results (2026-10-20)

Check	Result
`cd app; python -m pytest tests -q`	**31 passed** (unit, end-to-end, GUI offscreen, demo-dataset GUI)
`cd website; python -m pytest test_build.py -q`	**8 passed**
`cd website; python build.py; python check_site.py`	16 pages, 363 internal refs, 1 download link: **OK**
`python tools/build_release.py`	Frozen `--selftest` **OK**; GUI launcher alive after 10 s; zip test OK
Artifact	`Epoch-0.1.0.dev0-win-x64.zip`, 238,311,200 bytes, SHA-256 `36df3bca73d855908d1f1b3702293ec75069987e88348465724ee7976419ba1e` (about 529 MB unpacked)
Served copy in `website/dist/downloads`	SHA-256 identical to the manifest

5. Known defects and limitations

1. Dev zip is unsigned (SmartScreen warning) and was tested only on the build machine; not on a clean Windows install or Windows 10.
2. The windowed launcher shows no error output; use the CLI launcher to diagnose.
3. Exported models are joblib/pickle files. Loading requires an explicit trust flag; the manifest detects modification, not malicious origin.
4. Leakage probe is a heuristic. The selected model is not refit on train+val.
5. `evaluate_test` and `export` re-read the CSV on the calling thread (the first read is cached only after import or preview).
6. joblib emits a numpy 2.5 DeprecationWarning (harmless today).
7. GUI was inspected through screenshots rendered by Qt's native Windows platform and through automated tests; it was not clicked through by hand, and high-DPI and very small window sizes were not checked.
8. Website accessibility (contrast, keyboard, screen reader) was designed for but not audited with tools. Mobile overflow was checked only with a script and one visual check.
9. No version control (git is not installed). No CI.
10. The design PDF has not been revised to mark implemented vs planned, apart from its screen table.

6. Not done

Website deployment; git; code signing and a real installer; hyperparameter search, cross-validation, group and temporal splits; Excel and Parquet; GPU discovery; pause and resume; plugin API; safer export format (ONNX or skops); image, audio, text, time-series and neural domains.

7. Key decisions

- PySide6 for the UI; training in a spawned worker process for isolation and real cancellation.
- Test split sealed: `Workspace.evaluate_test` is single-use unless `acknowledge_repeat=True`; every use is logged.
- Fixed default hyperparameters; selection by validation macro-F1 or RMSE.
- XGBoost dropped: a custom wrapper would make bundles depend on this package.
- The release manifest is edited by hand (or a one-off script) from the real built file, never by the build script.

8. Next recommended actions

1. Install git and commit. 2. Test the zip on a clean Windows machine and fix what breaks. 3. Deploy `website/dist` (host the 238 MB zip as a release asset and switch the manifest entry to a verified `url`). 4. Group-aware and temporal splits, cross-validation. 5. Safer export format. 6. Code signing, installer, CI. 7. Hand-click the GUI at high DPI and small sizes; audit website accessibility.